

Model interface

A hands-on course in building agents by building their harness

Lesson 1

The model is a function

TokensOut = Model(TokensIn)

- Tokens go in as a **request**; tokens come back as a **response**
- Just like any other API endpoint
- The **model interface** is how the harness makes that call

The model interface is the parentheses

Agent = Harness(Model)

- **Control flow** decides when to call
- **Input** gathers what goes in
- **Context** decides how it's presented in the request
- **Output** handles the response

The concept (abridged)

```
from anthropic import Anthropic

client = Anthropic()

request = {
    "model": "claude-sonnet-5-5",
    "max_tokens": 16384,
    "messages": [{"role": "user", "content": "What's in this directory?"}],
}
output = client.messages.create(**request)

for block in output.content:
    print(f"[{block.type}]", getattr(block, "text", ""))
print("stop_reason:", output.stop_reason)
print("tokens in:", output.usage.input_tokens)
print("tokens out:", output.usage.output_tokens)
```

What's in it

- `client = Anthropic()` : where the request goes, Anthropic's hosted API through the official SDK
- `request` : the `model`, the tokens in as `messages` (each with a `role` and `content`), and `max_tokens`
- `max_tokens` : caps the tokens out, even mid-sentence
- `client.messages.create(...)` : sends it and waits; the whole response comes back at once
- `output` : the response, the tokens out plus how they were produced

Run it

```
uv run lessons/01-model-interface/model_interface.py
```

```
[thinking]  
stop_reason: end_turn  
tokens in: 15  
tokens out: 239
```

One real run (shortened: the `text` line is on the next slides)

Three fields matter

- **content** : the tokens out, as a list of blocks: first its `thinking` , then the `text` of its reply
- The thinking printed empty: by default it comes back with no text
- **stop_reason** : `end_turn` means it finished; `max_tokens` means it hit the cap
- **usage** : `input_tokens` 15 is TokensIn, `output_tokens` 239 is TokensOut, thinking included

It knows the next step. It can't take it.

The `text` block, in part:

To see what's in a directory yourself, you can use:

macOS/Linux: `ls` (basic), `ls -la` (includes hidden files and details)

quark.py: the same call, streamed (abridged)

```
from anthropic import Anthropic

# — model interface —————
client = Anthropic()
MODEL = "claude-sonnet-5-5"
def call(each=lambda event: None, **request):
    with client.messages.stream(model=MODEL, **request) as stream:
        for event in stream: each(event)
    return stream.get_final_message()

output = call(max_tokens=16384, messages=[...])
print(output.model_dump_json(indent=2))
```

The `messages` list holds one `user` message: "What's in this directory?"

One function, one place

- `client = Anthropic()` : the same as before
- `MODEL` : the model that answers, taken out of the request so every call sends the same one
- `call()` : the model interface, all of it. Every later lesson calls the model through it
- `messages=[...]` and `max_tokens` : the request, passed through untouched
- `output` : the same kind of response `create()` returned, printed whole

quark's choice: the response streams back

- `client.messages.stream(...)` : the response arrives piece by piece, as it's produced
- A long response that arrives all at once can time out while you wait; one that streams can't
- `for event in stream: each(event)` : each piece goes to `each()` as it arrives; here, `each` ignores them
- `stream.get_final_message()` : the whole response, put back together

Run quark.py

```
uv run lessons/01-model-interface/quark.py
```

```
{  
  "content": [  
    {  
      "thinking": "",  
      "type": "thinking"  
    },  
  ],  
}
```

One real run (shortened): the whole response is printed; here, its `thinking` block

Other things we could do

- **Where the request goes, and how:** hosted API, your own machine, a gateway; SDK or raw HTTP
- **Which model answers:** one model, or a backup when the first is unavailable
- **How the response arrives:** all at once like `model_interface.py`, or streamed like `quark.py`
- **When a request fails:** retry, wait out a rate limit, give up after a timeout
- **Settings and volume:** tokens out, how much it thinks, one request or a batch

A few worth knowing

- **Timeouts and retries:** the SDK does both; a brief hiccup costs a few seconds, not the run
- **A backup model:** if every retry fails, send the same request to the next model
- Catch only "try again later": dropped connections, rate limits, server errors, `OverloadedError`
- **Effort and thinking:** how much it thinks first, and a readable summary of that thinking

A gateway is this primitive alone

- [LiteLLM](#) and [OpenRouter](#) are the model interface and nothing else
- A harness sends them one request format
- They handle where it goes, which provider and model answer, backups, retries and rate limits
- A harness that uses one has handed off its model interface

The rule

The model interface sends tokens to the model as a **request** and gets tokens back as a **response**.

What it never does

- When to call is **control flow**
- Gathering what goes in, from a person or the world, is **input**
- How it's presented in the request is **context**
- What happens to the response is **output**

The model interface only gets the request there and the response back.

Both ends are stubs

```
"What's in this directory?" → request → model interface → response → print(...)
input (a stub)                                     output (a stub)
```

- The input is a hardcoded string no one typed
- The output dumps the whole response without handling any of it
- The model said what to do, and nothing could do it

Next: Input and output

A hands-on course in building agents by building their harness

Lesson 2